Provenance-Discounted Hierarchical Reinforcement Learning for Agentic Software Engineering

Anonymous Authors
Anonymous Institution
anonymous@institution.edu

Abstract

The application of Reinforcement Learning (RL) to Large Language Models (LLMs) for long-horizon agentic tasks, such as software engineering, is heavily bottlenecked by sparse rewards, data scarcity, and exploration collapse. When an agent fails to solve a complex coding task, a reward of zero provides no actionable gradient for policy optimization. To address these challenges, we introduce a novel training architecture that decouples planning from execution and leverages a human-grounded dynamic hinting curriculum. Furthermore, to balance the high quality of scarce human demonstrations with the high volume of synthetic LLM data, we propose *Provenance-Discounted Reward Shaping*. By dynamically penalizing reliance on environmental hints and capping maximum rewards based on data provenance, our framework seamlessly integrates Group Relative Policy Optimization (GRPO) to teach LLM agents robust, human-aligned software engineering capabilities without succumbing to model collapse or reward hacking.

1 Introduction

The transition from text-completion Large Language Models (LLMs) to autonomous, tool-using agents represents a paradigm shift from single-turn Markov Decision Processes (MDPs) to temporally extended, Partially Observable Markov Decision Processes (POMDPs). In the domain of software engineering (SWE), training an agent via standard Reinforcement Learning from Human Feedback (RLHF) often fails. This is due to the *Exploration Collapse* problem: long-horizon tasks have sparse, binary outcome rewards (e.g., unit tests pass or fail). An agent taking 30 steps to solve a GitHub issue will likely fail during exploration, receive a reward of 0.0, and learn nothing.

Recent literature, such as Agent-RLVR and ARE, has explored ”Teacher-Student” pedagogical guidance, wherein the environment injects hints to prevent the agent from getting stuck. Concurrently, decoupled planning frameworks like RLTR have demonstrated that explicitly optimizing a macro-plan before execution improves downstream action sequences.

This paper synthesizes these breakthroughs into a unified architecture. We propose a two-objective hierarchical RL setup combined with a *Human-Grounded Hinting Pipeline* and *Provenance-Discounted Reward Shaping*. This framework provides dense, step-level credit assignment while enabling scalable training on mixed datasets of human ground-truth and synthetic LLM rollouts.

2 Hierarchical, Guidance-Discounted RL

We split the cognitive load of the agent into two distinct optimization objectives: a macro-objective for strategic planning and a micro-objective for execution with dynamic guidance.

2.1 Macro-Objective: The Planning Phase

Inspired by RLTR (Reinforcement Learning with Tool-use Rewards), we decouple the reasoning and execution phases to prevent an imbalanced optimization objective. At the start of an episode, the agent is constrained to output a `<PLAN>` block, decomposing the complex software engineering ticket into discrete sub-tasks (e.g., 1. Reproduce the bug, 2. Locate the file, 3. Edit logic, 4. Test).

We employ a Process Reward Model (PRM) to score the quality and verifiable completeness of this plan independently of the final outcome. A logically sound plan receives an immediate positive reward, isolating planning capability from execution failures.

2.2 Micro-Objective: Execution via Fallback Training Loop

To overcome the sparse reward problem during execution, we implement a dynamic "Hinting Curriculum" inspired by human scaffolding. This guarantees the agent will eventually find a successful trajectory, creating a "golden trajectory" from which the RL algorithm can learn.

The execution loop operates on a sliding scale of autonomy, with rewards discounted heavily as environmental intervention increases:

- **Attempt 1 (No Hint):** Independent execution. Success yields the Maximum Reward ($R = 1.0$).
- **Attempt 2 (High-Level Hint):** The environment injects a conceptual nudge. Success yields a Discounted Reward ($R = 0.6$).
- **Attempt 3 (Low-Level Hint):** The environment provides explicit, step-by-step instructions. Success yields a Heavily Discounted Reward ($R = 0.2$).
- **Failure:** Inability to solve the task even with low-level hints results in $R \leq 0.0$.

Because of the reward discounting hierarchy, policy optimizers like GRPO or PPO naturally push the model's policy to maximize expected returns by relying less on hints over time, thereby incentivizing autonomy.

3 The Human-Grounded Hinting Pipeline

A naive implementation of the hinting curriculum relies on a massive, dynamic LLM oracle (e.g., GPT-4) to generate hints at runtime. This introduces prohibitive computational latency and risks "hallucinated" hints that cause the student model to fail. We propose an offline-to-online distillation approach.

3.1 Offline Hint Distillation

We pre-compute hints offline using a dataset of solved GitHub issues (e.g., SWE-bench). An orchestrator LLM analyzes the human-merged Pull Request (the exact code diff) and reverse-engineers the human's thought process into a curriculum of three hints of increasing explicitness. These hints are stored statically in a training database.

3.2 Fast RL Training Loop

During the reinforcement learning phase, the environment performs an $O(1)$ database lookup to inject the pre-computed human hint when the agent fails a unit test. This yields three structural advantages:

1. **Zero Runtime Hallucinations:** The hints are mathematically grounded in the verified human diff.
2. **Massive Compute Savings:** Rollout bottlenecks are eliminated as no heavy LLM inference is required to generate the environment's feedback.
3. **Idiomatic Alignment:** The agent is implicitly guided toward human-like software engineering patterns, mitigating "spaghetti code" shortcuts.

4 Provenance-Discounted Reward Shaping

To scale RL without suffering from model collapse, we must balance the high quality of human data with the high volume of synthetic data. We achieve this through Provenance-Discounted Reward Shaping.

4.1 The Data Mixture

The training curriculum dynamically mixes two datasets:

- **Dataset A (Golden Anchor):** Real GitHub issues solved by humans (e.g., 20% of the mix). Hints are distilled from the actual human PR. The maximum allowable reward is $R_{max} = 1.0$.
- **Dataset B (Volume Boilerplate):** Synthetic coding issues and solutions generated by an orchestrator LLM (e.g., 80% of the mix). The maximum allowable reward is capped at $R_{max} = 0.6$.

4.2 Optimization Dynamics and Safeguards

Using Group Relative Policy Optimization (GRPO) or Group-in-Group Policy Optimization (GiGPO), the algorithm calculates advantages based on the expected returns. By capping synthetic task rewards at 0.6, the gradients heavily penalize the model for adopting the lazy or hallucinated behaviors typical of synthetic LLM code.

To prevent the agent from learning a bifurcated policy, **Blind Testing** is strictly enforced; the prompt format remains identical across both datasets, preventing the agent from identifying the provenance of the task. Consequently, the agent utilizes the high-volume Dataset B to master basic tool-use mechanics, while strictly aligning its high-level reasoning and coding style to the human-level standards of Dataset A.

5 Conclusion

This paper outlines a state-of-the-art architectural framework for training agentic LLMs. By decoupling the macro-planning objective from micro-execution, and injecting human-distilled scaffolding into the environment, we solve the exploration collapse problem inherent to long-horizon RL. Furthermore, Provenance-Discounted Reward Shaping allows for massive data scaling via synthetic datasets without sacrificing the idiomatic quality of human ground truth. This synthesis of techniques paves the way for highly autonomous, self-improving software engineering agents.